



Индивидуальный проект

Дисциплина: Языки
программирования

Выполнил: Горбаненко К. Д.

Группа: ИТ-3 | Вариант 2

ПОСТАНОВКА ЗАДАЧИ: ИГРА В СЛОВА

Цель: Выстроить набор слов в цепочку по правилам игры.

Каждое следующее слово начинается на букву, которой закончилось предыдущее.

Последнее слово цепочки замыкается на первую букву первого слова.



ОСОБЕННОСТИ И ОГРАНИЧЕНИЯ



Правило 'Б'

Если слово кончается на «Ь», используется предыдущая буква.



Цикличность

Цепочка должна быть замкнутой (кольцо).



Реализация

Использование собственных динамических структур данных.

СТРУКТУРЫ ДАННЫХ

Класс Node: хранит слово и ссылку на следующий узел.

Класс WordList: методы `append()`, `to_list()`, подсчет размера.

Это соответствует требованию об использовании собственных классов вместо стандартных контейнеров.

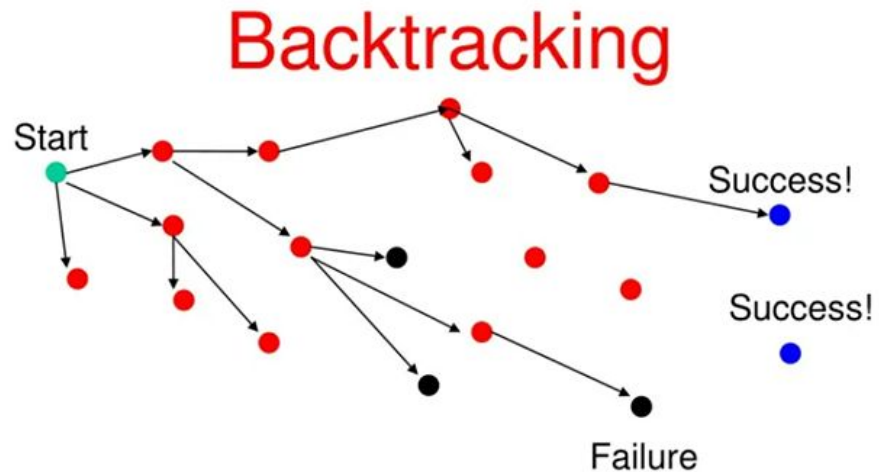
```
class Node:
    def __init__(self, word: str):
        self.word = word
        self.next: "Node | None" = None

class WordList:
    def __init__(self):
        self.head: Node | None = None
        self.size: int = 0

    def append(self, word: str) -> None:
        node = Node(word)
        if self.head is None:
            self.head = node
        else:
            current = self.head
            while current.next is not None:
                current = current.next
            current.next = node
        self.size += 1

    def remove(self, word: str) -> bool:
        if self.head is None:
            return False
        if self.head.word == word:
            self.head = self.head.next
            self.size -= 1
            return True
        current = self.head
        while current.next is not None:
            if current.next.word == word:
```

АЛГОРИТМ РЕШЕНИЯ



Метод Backtracking

1. Пробуем каждое слово как начало.
2. Рекурсивно ищем подходящее следующее слово.
3. Если зашли в тупик — возвращаемся и пробуем другую ветку.
4. Проверяем условие цикла в конце.

ТЕСТИРОВАНИЕ: ПРИМЕР 1

Вход: хмель мороз налог лимон запах дым лошадь холм гол молох

Процесс: хмель (кончается на 'л') → лимон (начинается на 'л')...

Результат: хмель лимон налог гол лошадь дым мороз запах холм молох

Программа успешно находит одно из возможных решений и выводит его в консоль.

```
Введите слова через пробел (строчные буквы русского алфавита):  
>>> хмель мороз налог лимон запах дым лошадь холм гол молох  
  
Слова (10 шт.): хмель мороз налог лимон запах дым лошадь холм гол молох  
Поиск цепочки...  
  
Результат:  
хмель лимон налог гол лошадь дым мороз запах холм молох
```

ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Реализовано циклическое консольное меню:

- Ручной ввод слов.
- Загрузка из текстового файла.
- Сохранение результата.
- Правила задачи и выход

Главное меню:

1. Ввести слова вручную
2. Загрузить слова из файла
3. Сохранить последний результат в файл
4. Показать правила задачи
0. Выход

Выберите пункт меню:

ОБРАБОТКА ОШИБОК



Файлы

Проверка наличия
файла и прав доступа

```
Выберите пункт меню: 2
Введите путь к входному файлу:
>>> 1
▲ Файл «1» не найден.
```



Ввод

Защита от пустых строк и
некорректных символов.

```
Выберите пункт меню: !
▲ Неверный пункт меню. Введите число от 0 до 4.
```



Логика

Корректный вывод "No",
если цепочка
невозможна.

```
Слова (3 шт.): кот рог вот
Поиск цепочки...
Результат: No (решение не найдено)
```


РЕЗУЛЬТАТЫ РАБОТЫ

- ✓ Код соответствует стандарту PEP 8.
- ✓ Реализованы собственные классы динамических структур.
- ✓ Обеспечен дружелюбный интерфейс.

ЗАКЛЮЧЕНИЕ

Разработанное программное обеспечение позволяет эффективно решать лингвистические задачи по поиску цепочек слов.

Использование алгоритма поиска с возвратом гарантирует нахождение решения, если оно существует, при соблюдении всех заданных условий (ь-правило, цикличность).

Спасибо за внимание!

Готов ответить на ваши
вопросы.